

Project 1: Automate Docker built and push using Jenkins file.

1. Setup a Simple Flask App

Project Structure : app.py has main Flask Application File.

```
Code Blame 10 lines (7 loc) · 162 Bytes

1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def hello_world():
7      return 'Hello, World!'
8
9  if __name__ == '__main__':
10     app.run(debug=True)
```

2. Docker File: Defines the Docker image for the Flask App.

```
Code Blame 20 lines (14 loc) · 518 Bytes

1  # Use an official Python runtime as a parent image
2  FROM python:3.9-slim
3
4  # Set the working directory in the container
5  WORKDIR /app
6
7  # Copy the current directory contents into the container at /app
8  COPY . /app
9
10 # Install any needed dependencies
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Make port 5000 available to the world outside this container
14 EXPOSE 5000
15
16 # Define environment variable to avoid Python buffering
17 ENV PYTHONUNBUFFERED 1
18
19 # Run app.py when the container launches
20 CMD ["python", "app.py"]
```

3. Jenkinsfile : Contains the Jenkins Pipeline Configuration.

```
Code Blame 29 lines (25 loc) · 709 Bytes

1  pipeline {
2      agent any
3
4      environment {
5          DOCKER_IMAGE = 'santhosh2010/my-flask-app:latest'
6      }
7
8      stages {
9          stage('Clone Repository') {
10             steps {
11                 git url: 'https://github.com/Santhosh2010/easy-docker.git', branch: 'main'
12             }
13
14             stage('Build Docker Image') {
15                 steps {
16                     sh 'docker build -t $DOCKER_IMAGE .'
17                 }
18             }
19
20             stage('Push Docker Image') {
21                 steps {
22                     withDockerRegistry(credentialsId: 'docker-hub-credentials', url: 'https://index.docker.io/v1/') {
23                         sh 'docker push $DOCKER_IMAGE'
24                     }
25                 }
26             }
27         }
28     }
29 }
```

4. Requirements.txt List of Dependencies (Flask and other)



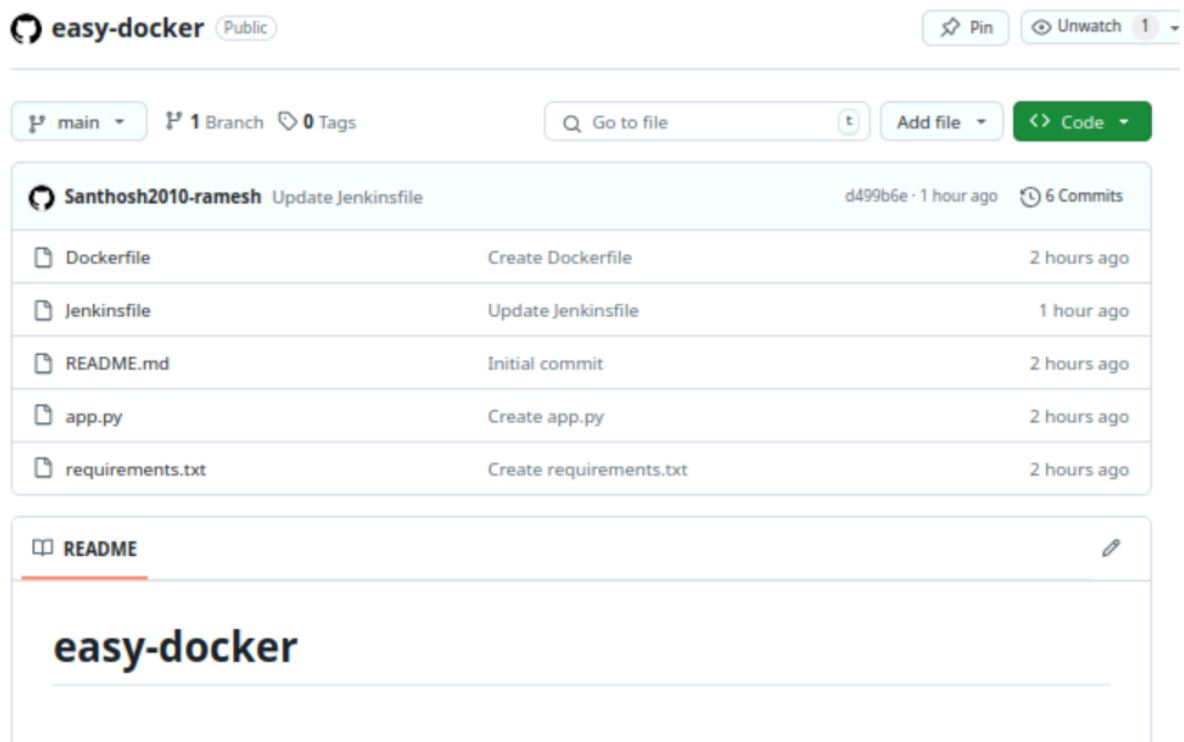
A screenshot of a GitHub code viewer for a file named `requirements.txt`. The interface shows the file is 2 lines long and 29 bytes. The code content is as follows:

```
1 Flask==2.2.2
2 Werkzeug==2.2.2
```

5. Push the code to GitHub

Make sure you have a GitHub repository created for the project.

Push all the files (app.py, requirements.txt, Docker File, Jenkins File) to the GitHub repository.



A screenshot of a GitHub repository named `easy-docker` by user `Santhosh2010-ramesh`. The repository is public and has 6 commits. The commit history shows the following files and actions:

File	Action	Time
Dockerfile	Create Dockerfile	2 hours ago
Jenkinsfile	Update Jenkinsfile	1 hour ago
README.md	Initial commit	2 hours ago
app.py	Create app.py	2 hours ago
requirements.txt	Create requirements.txt	2 hours ago

The README file is visible, showing the repository name `easy-docker`.

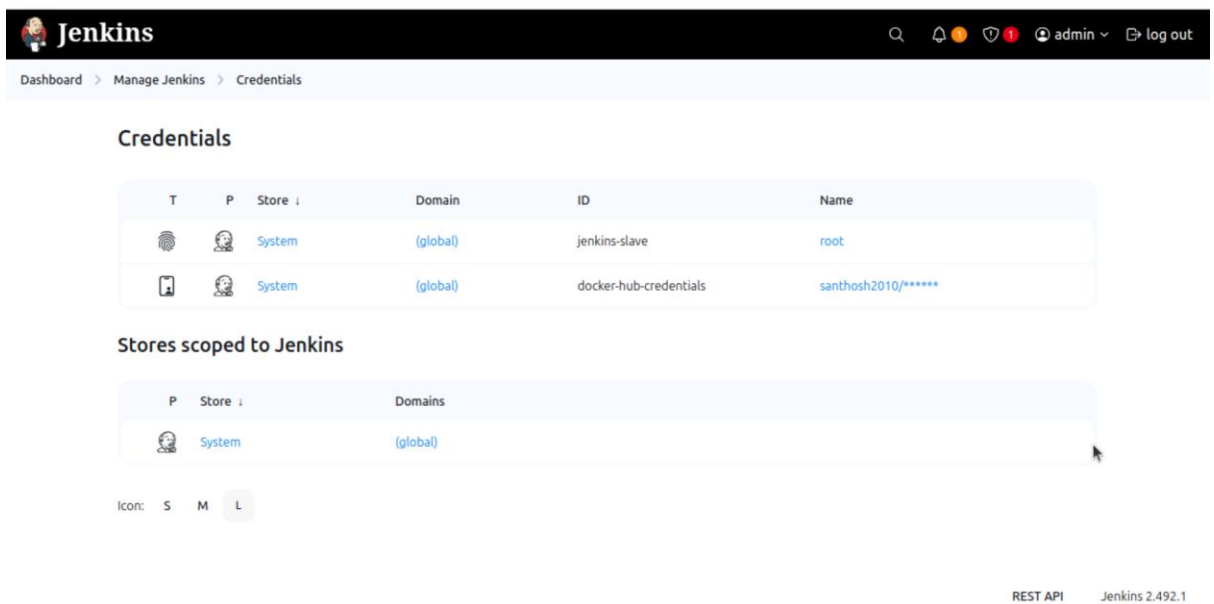
6. Configure Docker Hub Credentials in Jenkins

Goto Jenkins > manage Credentials.

Add new Credentials:

Username & Password: You Docker Hub username and password.

ID: Name it something like dockerhub-creds(the same name used in the Jenkinsfile)



The screenshot shows the Jenkins 'Credentials' page. At the top, there's a navigation bar with 'Dashboard > Manage Jenkins > Credentials'. Below this, the 'Credentials' section contains a table with two entries:

T	P	Store	Domain	ID	Name
		System	(global)	jenkins-slave	root
		System	(global)	docker-hub-credentials	santhosh2010/*****

Below the table is the 'Stores scoped to Jenkins' section, which shows a single entry:

P	Store	Domains
	System	(global)

At the bottom right, it says 'REST API' and 'Jenkins 2.492.1'.

7. Create a New Pipeline in Jenkinsfile

In Jenkinsfile, click New Item > Pipeline.

Enter a new for the Pipeline.

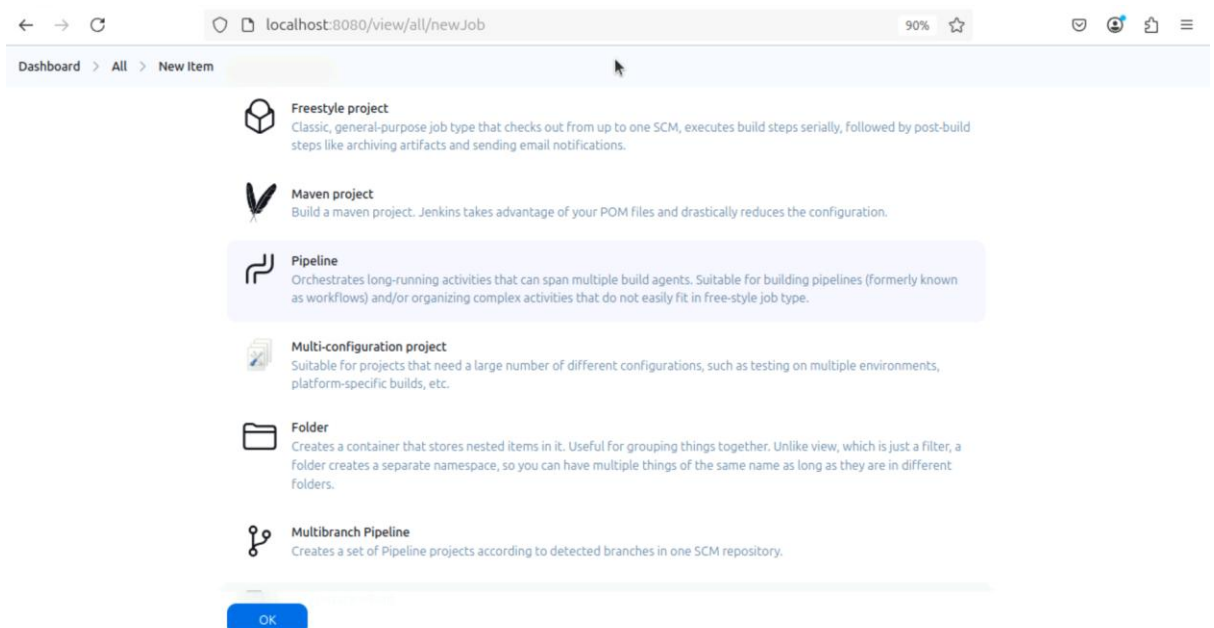
Under Pipeline definition, Select Pipeline Script from SCM.

Select Git as the SCm.

Enter the Github repository URL.

Set the branch (typically master or main)

Click Save



The screenshot shows the Jenkins 'New Item' page. The browser address bar shows 'localhost:8080/view/all/newJob'. The page has a navigation bar with 'Dashboard > All > New Item'. Below this, there are several project type options:

- Freestyle project**: Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**: Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Pipeline**: Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type. (This option is highlighted with a blue background.)
- Multi-configuration project**: Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**: Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**: Creates a set of Pipeline projects according to detected branches in one SCM repository.

At the bottom, there is a blue 'OK' button.



Jenkins

🔍

🔔

🛡️

👤 admin

🚪 log out

Dashboard

>

Manage Jenkins

>

Credentials

Credentials

T	P	Store	Domain	ID	Name
		System	(global)	jenkins-slave	root
		System	(global)	docker-hub-credentials	santhosh2010/*****

Stores scoped to Jenkins

P	Store	Domains
	System	(global)

Icon:

S

M

L

T	P	Store	Domain	ID	Name
		System	(global)	jenkins-slave	root
		System	(global)	docker-hub-credentials	santhosh2010/*****

Stores scoped to Jenkins

P Store **Domains**

 System (global)

Icon: S M **L**

8. Click Build Now

Dashboard > build_docker > </> Changes

Build Now
 Configure
 Delete Pipeline
 Full Stage View
 Favorite
 Open Blue Ocean
 Stages
 Rename
 Pipeline Syntax

Builds
 Filter
 Today
 #3 10:10 AM
 #2 10:09 AM

Stage View

Average stage times:
(full run time: ~1min 3s)

	Declarative: Checkout SCM	Clone Repository	Build Docker Image	Push Docker Image
#3 10:10 No Changes	1s	1s	14s	9s
#2 10:09 No Changes	1s	1s	32s	26s
#1 09:57 No Changes	1s	1s	603ms	238ms
			failed	failed
			11s	1s
			failed	failed

Permalinks

- Last build (#3), 1 hr 41 min ago
- Last stable build (#3), 1 hr 41 min ago
- Last successful build (#3), 1 hr 41 min ago
- Last failed build (#2), 1 hr 42 min ago

Dashboard
build_docker
#3

Status

Changes

Console Output

Edit Build Information

Delete build '#3'

Timings

Git Build Data

Open Blue Ocean

Pipeline Overview

Pipeline Console

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console Output

Download
Copy
View as plain text

Started by user **admin**

Obtained Jenkinsfile from git <https://github.com/Santhosh2010-ramesh/easy-docker>

[Pipeline] Start of Pipeline

[Pipeline] node

Running on **Jenkins** in `/var/lib/jenkins/workspace/build_docker`

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Declarative: Checkout SCM)

[Pipeline] checkout

Selected Git installation does not exist. Using Default

The recommended git tool is: NONE

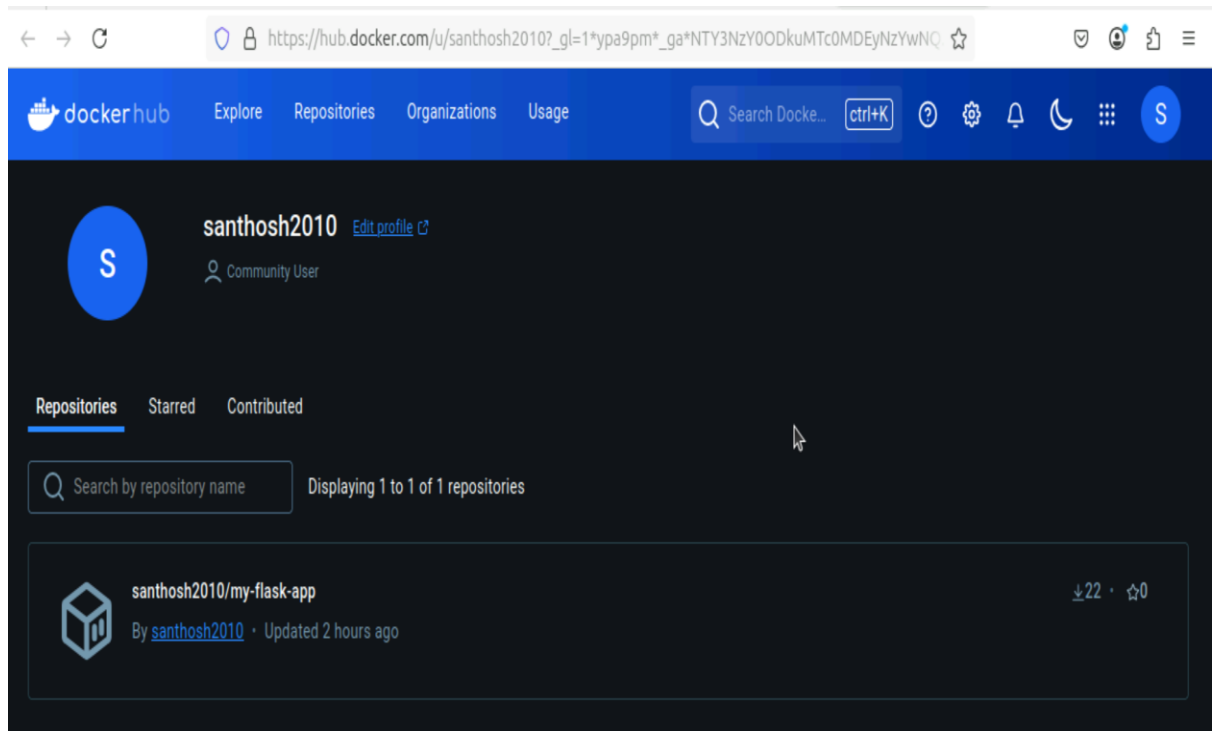
No credentials specified

```
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/build_docker/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Santhosh2010-ramesh/easy-docker # timeout=10
Fetching upstream changes from https://github.com/Santhosh2010-ramesh/easy-docker
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/Santhosh2010-ramesh/easy-docker +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision d499b6ef978ac80fb0ca6b59d2c57c0ff902059b (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f d499b6ef978ac80fb0ca6b59d2c57c0ff902059b # timeout=10
Commit message: "Update Jenkinsfile"
> git rev-list --no-walk d499b6ef978ac80fb0ca6b59d2c57c0ff902059b # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
```

9. Verify Docker Image on Docker Hub.

After the Build Finishes, log into your Docker Hub Account.

You should see the `my_flask_app` image under Repositories with the latest tag.



Project 2: Installing and enabling Docker Inside Ubuntu Terminal.

Step 1: Installing Nginx.

```
santhosh@ccb228d15332571: ~  
santhosh@ccb228d15332571:~$ docker --version  
Docker version 27.3.1, build ce12230  
santhosh@ccb228d15332571:~$ docker pull nginx  
Using default tag: latest  
latest: Pulling from library/nginx  
Digest: sha256:9d6b58feebd2dbd3c56ab585333d627cc6e281011cfd6050fa4bcf2072c9496  
Status: Image is up to date for nginx:latest  
docker.io/library/nginx:latest  
santhosh@ccb228d15332571:~$
```

Step 2: Displaying the Image List.

```
santhosh@ccb228d15332571:~$ docker images  
REPOSITORY TAG IMAGE ID CREATED SIZE  
nginx latest 9d6b58feebd2 3 weeks ago 279MB  
swarm latest 2de883e2933 4 years ago 16.6MB  
santhosh@ccb228d15332571:~$
```

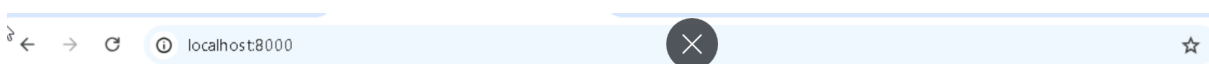
Step 3: Running the Container.

```
santhosh@ccb228d15332571:~$ sudo docker run -p 8000:80 nginx  
[sudo] password for santhosh:  
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration  
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh  
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf  
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf  
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
```

Step 4: Running the Renamed Container.

```
santhosh@ccb228d15332571:~$ sudo docker run --name renamed_nginx -d -p 8000:80 nginx  
ba53f165a3fb9e23cf8c7822684a36d7887c322b59cb78c1b1d22f6c44ff1c10
```

Step 5: Localhost:8000



Welcome to Nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.

Commercial support is available at nginx.com.

Thank you for using Nginx.

Step 6: Displaying the running container.

```
santhosh@ccb228d15332571:~$ sudo docker ps  
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES  
ba53f165a3fb nginx "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:8000->80/tcp renamed_nginx
```

Step 7: Interacting Directly with renamed Container.

```
santhosh@ccb228d15332571:~$ sudo docker exec -it renamed_nginx /bin/bash
root@ba53f165a3fb:/#
```

Step 8: Modifying the index.html file.

```
The following NEW packages will be installed:
 libgpm2 libncursesw6 nano
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 830 kB of archives.
After this operation, 3339 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
get:1 http://deb.debian.org/debian bookworm/main amd64 libncursesw6 amd64 6.4-4 [134 kB]
get:2 http://deb.debian.org/debian bookworm/main amd64 nano amd64 7.2-1+deb12u1 [690 kB]
get:3 http://deb.debian.org/debian bookworm/main amd64 libgpm2 amd64 1.20.7-10+b1 [14.2 kB]
Fetched 830 kB in 0s (6067 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package libncursesw6:amd64.
(Reading database ... 7580 files and directories currently installed.)
Preparing to unpack .../libncursesw6_6.4-4_amd64.deb ...
Unpacking libncursesw6:amd64 (6.4-4) ...
Selecting previously unselected package nano.
Preparing to unpack .../nano_7.2-1+deb12u1_amd64.deb ...
Unpacking nano (7.2-1+deb12u1) ...
Selecting previously unselected package libgpm2:amd64.
Preparing to unpack .../libgpm2_1.20.7-10+b1_amd64.deb ...
Unpacking libgpm2:amd64 (1.20.7-10+b1) ...
Setting up libgpm2:amd64 (1.20.7-10+b1) ...
Setting up libncursesw6:amd64 (6.4-4) ...
Setting up nano (7.2-1+deb12u1) ...
update-alternatives: using /bin/nano to provide /usr/bin/editor (editor) in auto mode
update-alternatives: warning: skip creation of /usr/share/man/man1/editor.1.gz because associated file /usr/share/man/man1/nano.1.gz (of link group editor) doesn't exist
update-alternatives: using /bin/nano to provide /usr/bin/pico (pico) in auto mode
update-alternatives: warning: skip creation of /usr/share/man/man1/pico.1.gz because associated file /usr/share/man/man1/nano.1.gz (of link group pico) doesn't exist
Processing triggers for libc-bin (2.36-9+deb12u9) ...
root@ba53f165a3fb:/# nano /usr/share/nginx/html/index.html
```

Step 9: Editing the line on Nginx Html file.

```
santhosh@ccb228d15332571: ~
GNU nano 7.2 /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Step 10: Nginx Localhost 8000 site.

